K J Somaiya College of Engineering, Mumbai

(Somaiya Vidyavihar University)

Department of Information Technology

Deep Learning - Mini Project Report

Academic Year 2026-27 (ODD Term) · LY B.Tech (IT) · Semester VII

SomaiyaSat Smart Monitor

Deep Learning for Satellite Health Monitoring and Earth Image Classification

AI Use Case: KJS-SRS-01 - SomaiyaSat & SomaiyaPod PocketQube Mission

Table 1: Group members

Name of Student	Roll Number	Batch
Yashveer Singh	16010423117	B2
Arjun Patil	16010423121	B2

Course: Deep Learning **Course In-charge:** Prof. Sarika Dharangaonkar

GitHub: https://github.com/arjunpatil-sudo/somaiyasat-smart-monitor

Live demo: https://somaiyasat-smart-monitor.vercel.app

Aim

Design, implementation, report submission, and demonstration of an AI use case related mini project (group of 2 students).

Abstract

SomaiyaSat is a PocketQube satellite developed under the KJS-SRS-01 AI use case. It continuously sends housekeeping telemetry (battery voltage, currents and temperatures) and captures Earth images for SSTV transmission. Analysing this data by hand is slow and error-prone. This project applies deep learning to automate two tasks: monitoring the satellite's health and classifying the images it captures. The work is built directly on the eight Deep Learning lab experiments.

For health monitoring, a physics-based simulator of the PocketQube power and thermal system generated 138,240 minutes of labelled telemetry, with five injected fault types. A back-propagation network written from scratch in NumPy, a feed-forward network and a deep network classify each reading as Normal, Warning or Critical, reaching up to **98.45%** accuracy on orbits held out from training. Recurrent (SimpleRNN) and LSTM models forecast battery voltage and temperature 15 minutes ahead from the previous hour. The LSTM reaches a mean absolute error of **19.0 mV** and **0.78 °C**, against 31.0 mV and 4.53 °C for a persistence baseline.

For image classification, a compact CNN (141,674 parameters, 553 KB of weights) trained from scratch on the EuroSAT Sentinel-2 dataset (27,000 images, 10 land-cover classes) achieves **96.54%** test accuracy. Its feature maps and Grad-CAM heat-maps are visualised to show what the network learns. Transfer learning with ImageNet-pretrained MobileNetV2 raises accuracy to **97.58%** ($\alpha = 1.0$). The $\alpha = 0.35$ variant reaches 95.53% with 423,018 parameters, which suits low-power on-board hardware. All models are compared on accuracy, loss, size and latency. They are deployed in a web dashboard where they run entirely in the browser, which shows that lightweight deep learning can support autonomous monitoring and smarter data handling on student-built satellites.

Keywords: PocketQube, telemetry, anomaly detection, back-propagation, deep neural network, CNN, feature maps, transfer learning, MobileNetV2, RNN, LSTM, EuroSAT.

Table of Contents

1. Introduction....4
1.1 Background....4
1.2 Problem statement....4
1.3 Objectives....4
1.4 Mapping of lab experiments to the project....4
2. Background and Related Work....6
3. Datasets....7
3.1 SomaiyaSat telemetry (simulated)....7
3.2 EuroSAT RGB....8
4. System Architecture and Tools....10
5. Implementation and Results....11
5.1 Experiment 1 - Back-propagation from scratch....11
5.2 Experiment 2 - Feed-forward neural network....11
5.3 Experiment 3 - Deep neural network....12
5.4 Experiment 5 - Convolutional neural network (EuroSAT)....14
5.5 Experiment 4 - Visualisation of feature maps....15
5.6 Experiment 6 - Transfer learning with MobileNetV2....17
5.7 Experiment 7 - Recurrent neural network....18
5.8 Experiment 8 - Long Short-Term Memory network....19
6. Comparative Analysis....21
7. Web Dashboard and Deployment....22
8. Conclusion and Future Scope....24
Limitations....24
Future scope....24
9. References....25
Appendix A - Project Structure and How to Run....26

1. Introduction

1.1 Background

PocketQubes are 5 cm-cube picosatellites built by universities and student teams. SomaiyaSat, together with the SomaiyaPod deployer, forms the KJS-SRS-01 mission. Like every spacecraft, it reports housekeeping telemetry (battery state, power-bus currents and board temperatures) and carries a small camera whose Earth images are sent to the ground as SSTV (slow-scan television) frames. A ground-station operator must decide on every pass whether the satellite is healthy and which images are worth keeping.

The satellite spends about one third of every ~95-minute orbit in Earth's shadow, so its battery, temperatures and power budget swing continuously. Spotting a problem early (a failing cell, a stuck heater, a latch-up on the power bus) gives operators time to act before the mission is lost. Similarly, tagging images automatically by what they show (forest, water, urban, farmland...) lets the team prioritise downlink time.

1.2 Problem statement

Build lightweight deep learning models that (a) classify SomaiyaSat's health from telemetry, (b) forecast critical telemetry channels ahead of time, and (c) classify the land type in Earth images. The models must be small enough to run on low-power hardware, and must be demonstrated in a working application.

1.3 Objectives

- Generate realistic PocketQube telemetry, with labelled fault episodes, for training and evaluation.
- Implement back-propagation from first principles and verify it numerically (Exp 1).
- Train feed-forward and deep neural networks for three-class health classification (Exp 2, 3).
- Train a CNN from scratch on EuroSAT and visualise its feature maps (Exp 4, 5).
- Apply transfer learning with MobileNetV2 and study the accuracy versus size trade-off (Exp 6).
- Forecast battery voltage and temperature with RNN and LSTM models (Exp 7, 8).
- Compare all models on accuracy, loss and model size, and deploy them in a web dashboard (GitHub + Vercel).

1.4 Mapping of lab experiments to the project

Table 2: Lab experiments and where each one is used

Exp	Lab experiment	CO	Role in this project
1	Back propagation algorithm	CO1	NumPy MLP health classifier with hand-coded gradients and a gradient check
2	Feed forward neural network	CO2	Single-hidden-layer FFNN health classifier
3	Deep neural network	CO2	Depth study and 128-64-32 BatchNorm/Dropout DNN (used on the dashboard)
4	Visualization of feature maps in CNN	CO3	First-layer filters, activation maps, Grad-CAM on Earth images

Exp	Lab experiment	CO	Role in this project
5	Convolutional neural network	CO3	CNN trained from scratch on EuroSAT (used on the dashboard)
6	Transfer learning with CNN	CO3	MobileNetV2 (α = 1.0 and 0.35) feature extraction and fine-tuning
7	Recurrent neural network	CO4	SimpleRNN 15-minute forecaster for voltage and temperature
8	Long short-term memory network	CO4	LSTM forecaster, compared with the RNN and a baseline (used on the dashboard)

2. Background and Related Work

Telemetry monitoring. Spacecraft operators traditionally use fixed limit checks on each channel. Data-driven methods instead learn the normal joint behaviour of many channels. Feed-forward and recurrent networks have been used for spacecraft anomaly detection, and LSTM-based forecasting of telemetry, with alerts raised on large prediction errors, was demonstrated on NASA's SMAP and MSL missions [5]. Our health classifier and forecasters follow the same two ideas at PocketQube scale.

Remote-sensing image classification. EuroSAT [3] provides 27,000 labelled 64×64 Sentinel-2 patches in 10 land-use and land-cover classes. Its authors report about 98.6% accuracy with a fine-tuned ResNet-50. CNNs learn hierarchical features, from edges and colour blobs in early layers to textures and object parts deeper in the network, which feature-map visualisation and Grad-CAM [6] make visible.

Efficient CNNs and transfer learning. MobileNetV2 [4] uses depthwise-separable convolutions in inverted-residual blocks with linear bottlenecks, which cut computation by roughly an order of magnitude compared with standard convolutions. A width multiplier α shrinks every layer, giving a family of models for constrained devices. Re-using ImageNet features (transfer learning) lets small datasets reach high accuracy quickly.

Recurrent networks. A SimpleRNN keeps a hidden state that is updated at every time step, but its gradients vanish over long sequences. The LSTM [7] adds input, forget and output gates and a cell state, so information can flow across many time steps. This matters for telemetry, where the next 15 minutes depend on where the satellite is in its 95-minute orbit.

3. Datasets

3.1 SomaiyaSat telemetry (simulated)

Real SomaiyaSat downlink logs are not publicly available, so we wrote a physics-based simulator (src/telemetry_sim.py) of a 1S Li-ion PocketQube in a ~500 km low Earth orbit. It produces one sample per minute and models:

- a 94.6-minute orbit with 37% of each orbit in eclipse, and body-mounted solar panels on a tumbling satellite;
- battery state of charge integrated from (solar - load) current, and terminal voltage = OCV(SOC) + I·R_internal;
- first-order thermal models of the battery and the on-board computer, with a thermostatic survival heater;
- payload activity: camera captures (+85 mA) and 6-10 minute SSTV transmissions (+210 mA);
- sensor noise on every channel (ADC and thermistor noise).

Each of the 48 simulated two-day runs (2,880 minutes each, 138,240 samples in total) is either nominal or contains one injected fault: battery ageing (reduced capacity, higher resistance, leakage), heater failure, solar-string loss, power-bus latch-up, or a transmitter stuck at high power. The health label is computed from the true, noise-free state using operator limits (Table 3). The models only see the noisy readings, so the classes overlap near the limits, just as they do on a real satellite.

Table 3: Health labelling rules

Class	Rule (any condition)
Critical	V < 3.40 V, or T_batt > 52 °C, or T_batt < -6 °C, or T_board > 70 °C, or load > 430 mA
Warning	V < 3.62 V, or T_batt > 40 °C, or T_batt < 2 °C, or T_board > 58 °C, or load > 300 mA, or sunlit with solar < 35 mA
Normal	none of the above

Inputs (7): battery voltage, battery current, battery temperature, board temperature, solar current, load current and the sun-sensor flag. **Split by run**, not by row, so that test orbits are never seen during training: runs with run mod 6 < 4 are used for training (92,160 samples), run mod 6 = 4 for validation and run mod 6 = 5 for testing (23,040 samples each). Features are standardised with the training-set mean and standard deviation.

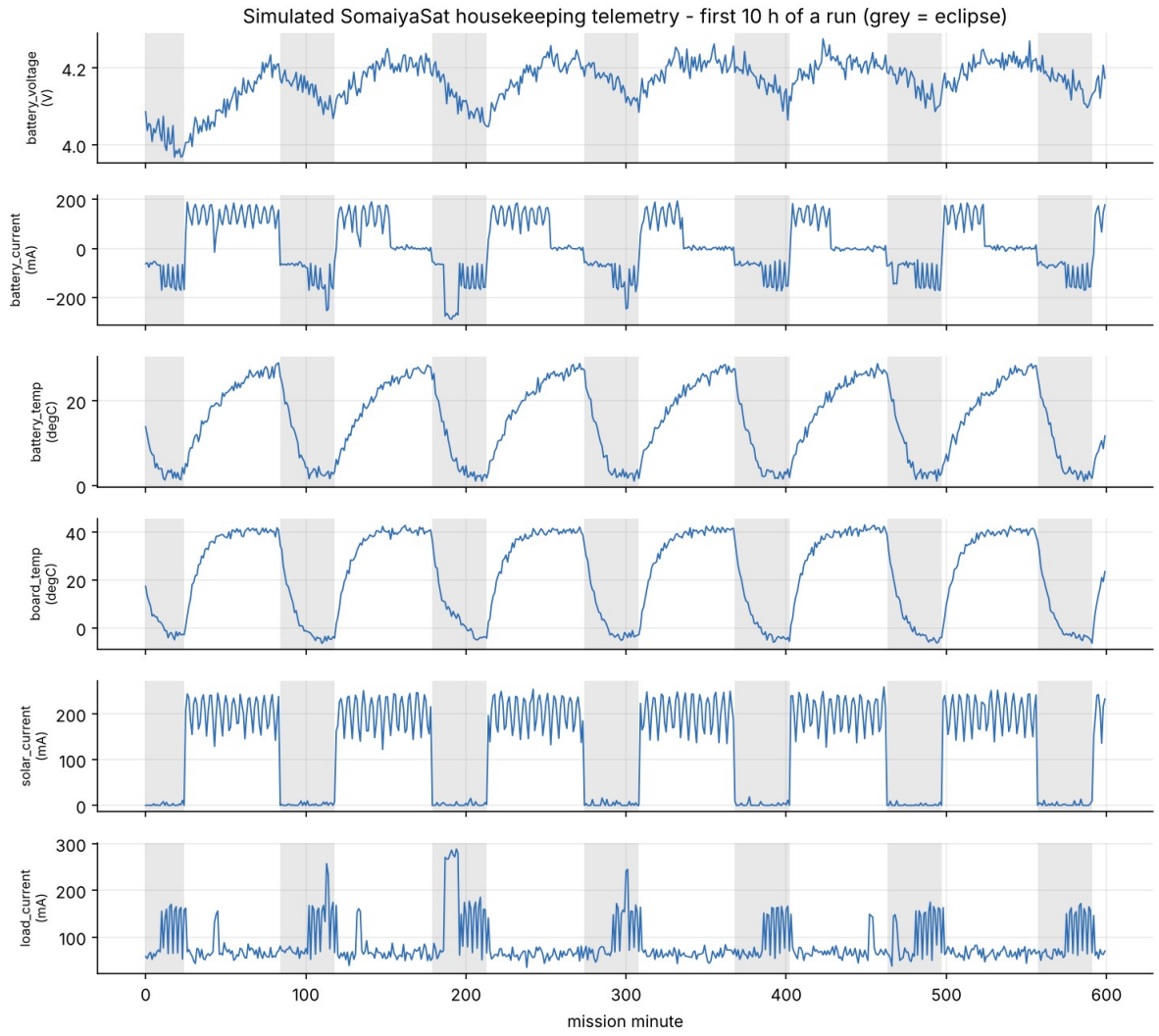


Figure 1: Ten hours of simulated telemetry (grey bands are eclipses). The heater can be seen cycling during eclipses.

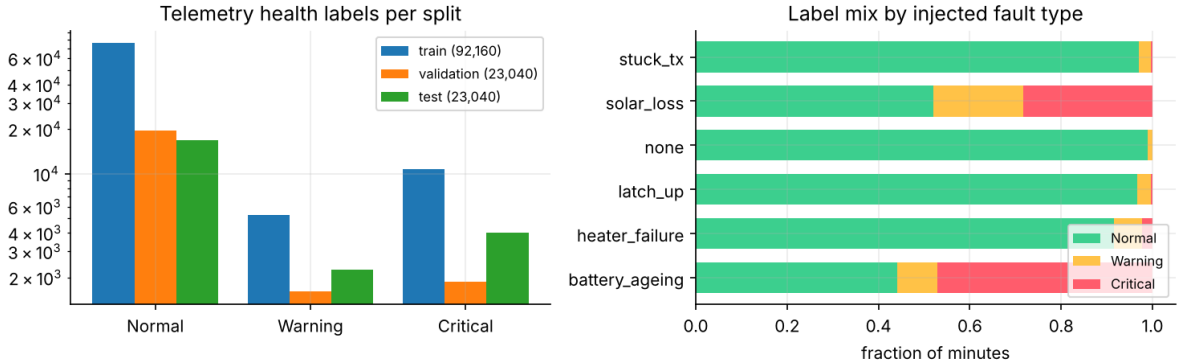


Figure 2: Class distribution per split (log scale) and the label mix for each fault type

3.2 EuroSAT RGB

EuroSAT RGB [3] (Zenodo record 7711810, EuroSAT_RGB.zip, 94.7 MB) contains 27,000 JPEG patches of 64×64 pixels at 10 m/pixel, from Sentinel-2 bands B4, B3 and B2. There are 10 classes with 2,000-3,000 images each: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River and SeaLake. It stands in for SomaiyaSat's Earth images. We used a stratified 70/15/15 split (seed 42): 18,900 training, 4,050 validation and

4,050 test images. Inputs are standardised per channel. Training uses random horizontal and vertical flips and 90° rotations, because satellite images have no fixed 'up'.

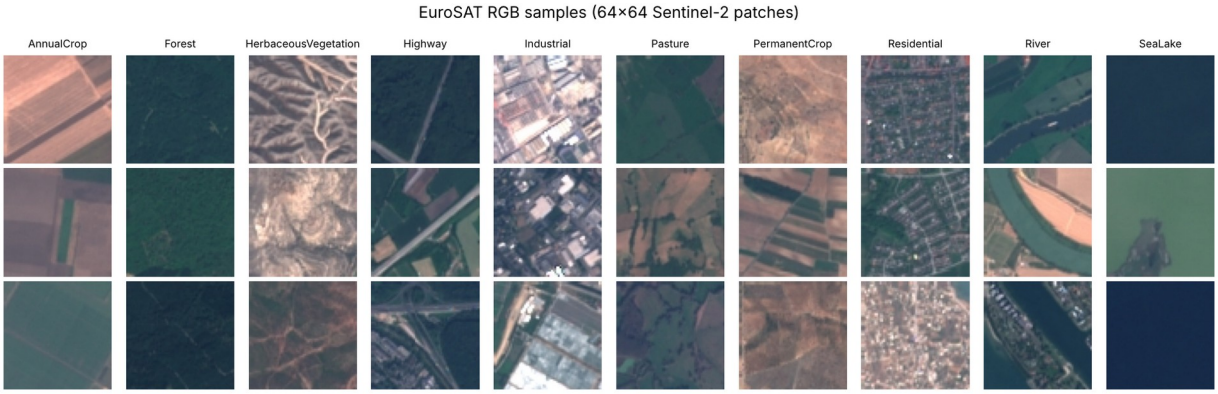


Figure 3: EuroSAT RGB samples, three per class

4. System Architecture and Tools

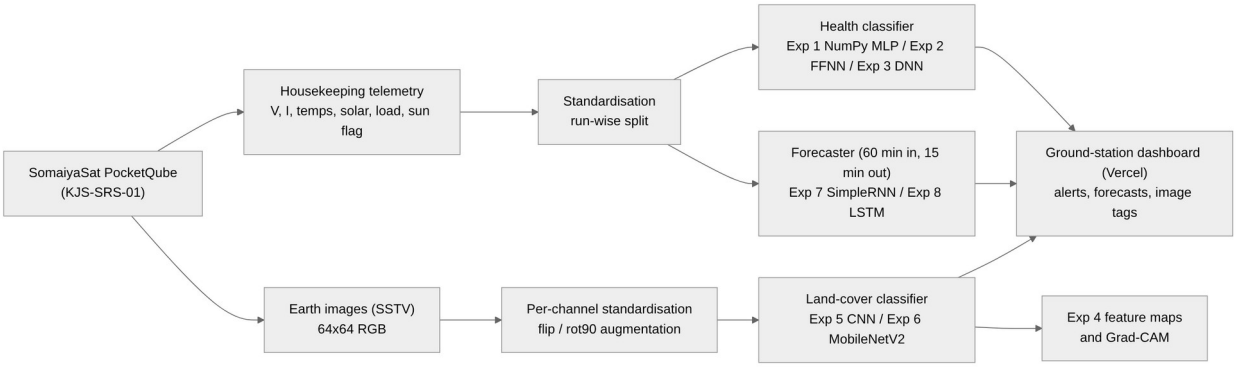


Figure 4: SomaiyaSat Smart Monitor: data flow from the satellite to the ground-station dashboard

The telemetry branch runs two models in parallel: a classifier that labels the current state, and a forecaster that predicts the next 15 minutes of the two channels most tied to mission loss (battery voltage and battery temperature). The image branch tags every downlinked image with a land-cover class. The trained weights are exported to JSON and binary files and executed in the browser by about 160 lines of plain JavaScript (dense, BatchNorm, LSTM, convolution and pooling layers), so the dashboard needs no server and could equally run on a ground-station laptop with no internet connection.

Table 4: Tools and technologies

Component	Technology
Language / DL framework	Python 3.13, TensorFlow 2.21 / Keras 3.15, NumPy (Exp 1)
Data and metrics	pandas, scikit-learn (splits, F1, confusion matrices), matplotlib
Pretrained model	MobileNetV2 ImageNet weights (keras-applications release files)
Dashboard	Static HTML/CSS/JavaScript, custom in-browser inference (web/js/nn.js)
Version control / hosting	Git + GitHub; Vercel (static deployment)
Hardware used for training	2-core CPU, 8 GB RAM (no GPU)

5. Implementation and Results

Every experiment is a standalone script in src/ (e.g. exp5_cnn.py). Each script writes its metrics to results/metrics/*.json and its figures to results/plots/. All numbers in this chapter are read from those files. Seeds are fixed (42) for reproducibility.

5.1 Experiment 1 - Back-propagation from scratch

A 7-16-3 MLP, tanh hidden, softmax output was implemented in NumPy only. The forward pass computes z1 = XW1 + b1, a1 = tanh(z1), z2 = a1W2 + b2, p = softmax(z2). The backward pass applies the chain rule by hand: dL/dz2 = p - y (softmax with cross-entropy), $dW2 = a1^{T} \cdot dz2$, $da1 = dz2 \cdot W2^{T}$, $dz1 = da1 \odot (1 - a1^{2})$, $dW1 = X^{T} \cdot dz1$. A class-weighted loss compensates for the dominance of Normal samples.

Gradient check. Before training, the analytic gradients were compared with central finite differences, (L(θ+ε) - L(θ-ε)) / 2ε with ε = 1e-5. The maximum relative error was **1.12e-8** (mean 5.27e-10), which confirms the derivation. Training used mini-batch SGD + momentum (lr=0.01, m=0.9, batch=128), for 40 epochs.

Table 5: Exp 1 results (held-out test runs)

Metric	Value
Parameters	179
Test accuracy	97.59%
Test macro-F1	0.9497
Training time	2.3 s

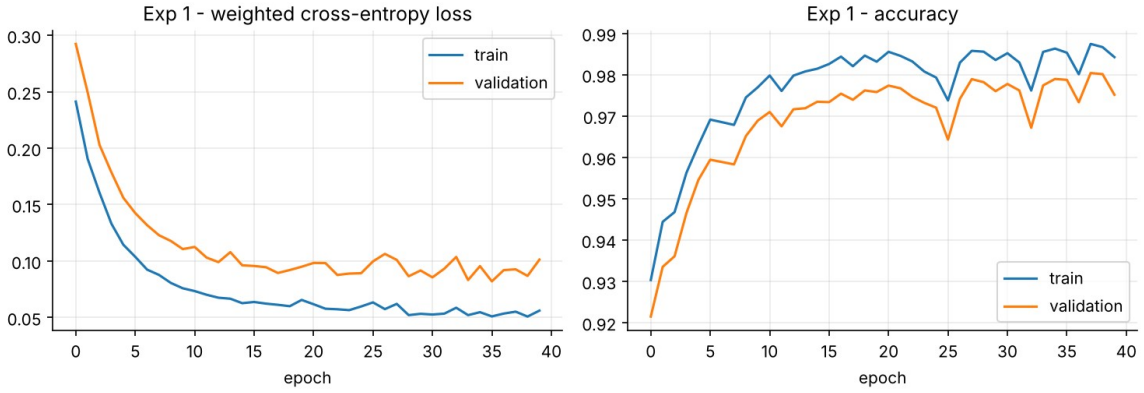


Figure 5: Exp 1 - loss and accuracy of the NumPy network

Observation: a network with only 179 parameters, trained with plain SGD + momentum, already separates the three health states well. Most of its errors are Normal readings flagged as Warning (false alarms) rather than missed critical events.

5.2 Experiment 2 - Feed-forward neural network

The same task was solved in Keras with a feed-forward network: 7 -> Dense(16, ReLU) -> Dense(3, softmax), trained with SGD + momentum 0.9, lr 0.05, batch 128, 40 epochs, class-weighted CE. Keras handles automatic differentiation, which replaces the hand-derived gradients of Exp 1.

Table 6: Exp 2 results

Metric	Value
Parameters	179
Test accuracy	98.09%
Test macro-F1	0.957
Weights (float32)	0.7 KB

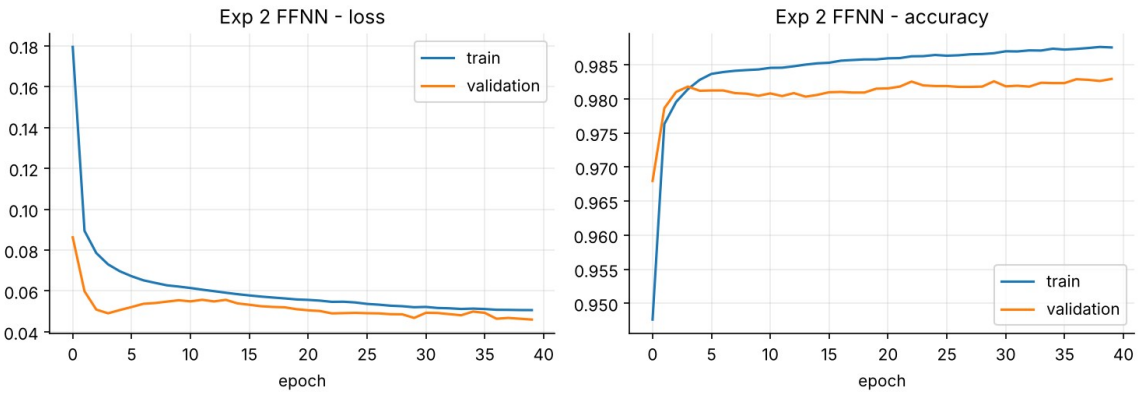


Figure 6: Exp 2 - FFNN training curves

5.3 Experiment 3 - Deep neural network

Depth study. Networks with 1 to 4 hidden layers of 64 ReLU units were trained with identical settings (Adam, 15 epochs).

Table 7: Exp 3 - effect of depth

Hidden layers	Parameters	Test accuracy	Macro-F1
1	707	96.85%	0.9376
2	4,867	97.68%	0.9512
3	9,027	97.93%	0.9552
4	13,187	98.20%	0.9584

Accuracy rises from 96.85% with one hidden layer to 98.20% with four. The extra layers model the interactions between channels: for example, low solar current is only abnormal when the sun sensor says the satellite is sunlit, and a given voltage is worrying or not depending on the load.

Regularisation study. The 128-64-32 network was then trained with different dropout rates, with and without BatchNorm, and with and without class-weighted loss (Table 8). Heavy dropout (0.2) and class weighting both pushed the model towards raising extra Warnings, which lowers accuracy. BatchNorm with light dropout (0.1) and an unweighted loss gave the best result.

Table 8: Exp 3 - regularisation study (128-64-32 network)

Dropout	BatchNorm	Class-weighted loss	Test accuracy	Macro-F1
0.2	yes	yes	97.58%	0.9487
0.1	yes	yes	97.75%	0.9521
0	yes	yes	98.11%	0.9587
0.1	no	yes	98.26%	0.9613

Dropout	BatchNorm	Class-weighted loss	Test accuracy	Macro-F1
0.1	yes	no	98.45%	0.9624

Final DNN. 7 -> [Dense128-BN-ReLU-Drop0.1] -> [Dense64-BN-ReLU-Drop0.1] -> [Dense32-BN-ReLU-Drop0.1] -> Dense3 softmax. Training used Adam lr 1e-3, batch 256, unweighted CE, early stopping (patience 8), ReduceLROnPlateau, and early stopping ended it after 46 epochs. BatchNorm stabilises training, and dropout reduces co-adaptation of neurons. This model runs in the dashboard's live health monitor.

Table 9: Exp 3 - final DNN results

Metric	Value
Parameters	12,131
Test accuracy	98.45%
Test macro-F1	0.9624
Critical recall	0.9831
Warning recall	0.8857
Weights (float32)	47 KB

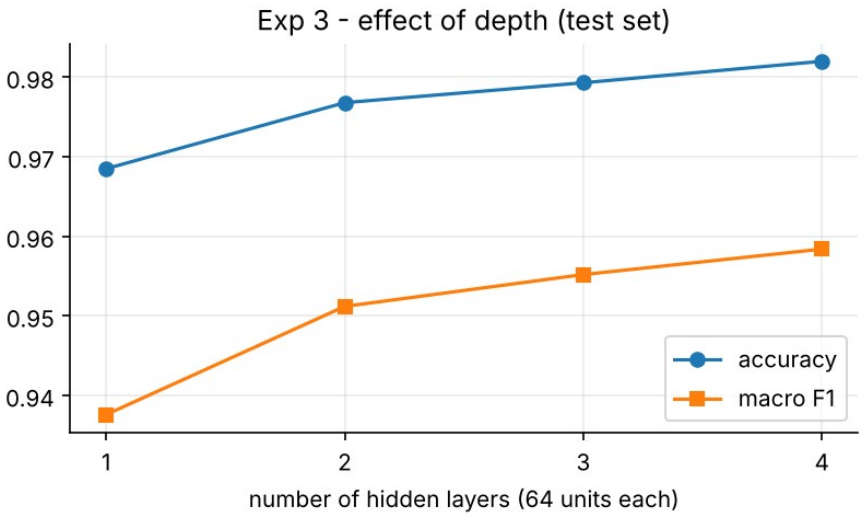


Figure 7: Exp 3 - test accuracy and macro-F1 against the number of hidden layers

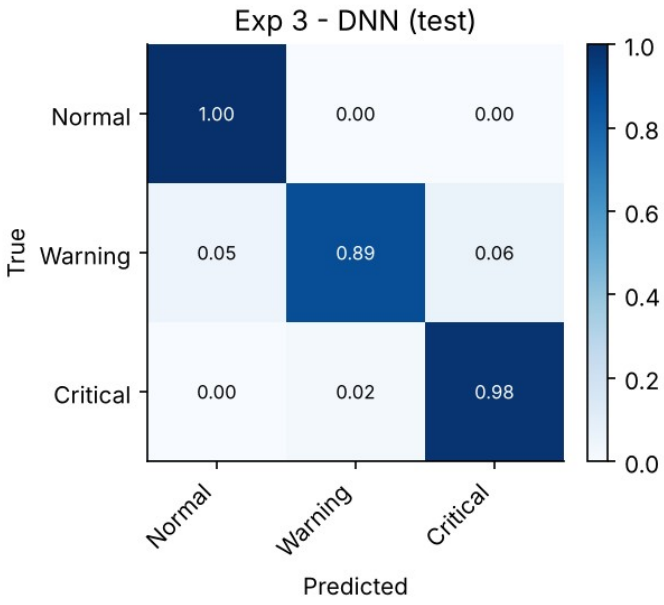


Figure 8: Exp 3 - normalised confusion matrix of the DNN on held-out runs

Observation: the DNN never confuses Critical with Normal in either direction. Every error is between neighbouring classes, which is expected because the noisy sensors make readings near a limit ambiguous. 98.31% of Critical readings are caught, and the rest are reported as Warning, so an operator is still alerted. Compared with the single-hidden-layer FFNN (Exp 2), the DNN raises far fewer false Warnings on Normal data (33 vs 178 test minutes).

5.4 Experiment 5 - Convolutional neural network (EuroSAT)

Experiment 5 is presented before Experiment 4 because Exp 4 visualises the network trained here.

Architecture: 2x[Conv32-BN-ReLU] Pool, 2x[Conv64-BN-ReLU] Pool, [Conv128-BN-ReLU] Pool, GAP, Dropout0.3, Dense10. All convolutions are 3×3 with 'same' padding and no bias; each is followed by BatchNorm. Global average pooling replaces large fully-connected layers, which keeps the model at only **141,674 parameters (553 KB of float32 weights)**. **Training:** Adam lr 1e-3, batch 64, ReduceLROnPlateau, early stopping; flip + rot90 augmentation; 22 epochs in 35 minutes on a 2-core CPU.

An issue we found and fixed: in our first attempt the inputs were only rescaled to [0, 1]. Because EuroSAT images are dark, first-layer activations had tiny variance, and BatchNorm's running averages lagged far behind the batch statistics. Validation accuracy collapsed to 13-47% while training accuracy rose to 84%. Standardising each input channel with the training-set mean and standard deviation, and using a BatchNorm momentum of 0.9, fixed this.

Table 10: Exp 5 - CNN results on 4,050 test images

Metric	Value
Test accuracy	96.54%
Test macro-F1	0.9646
Parameters	141,674
Weights (float32)	553 KB
CPU latency (batch = 1)	2.56 ms / image

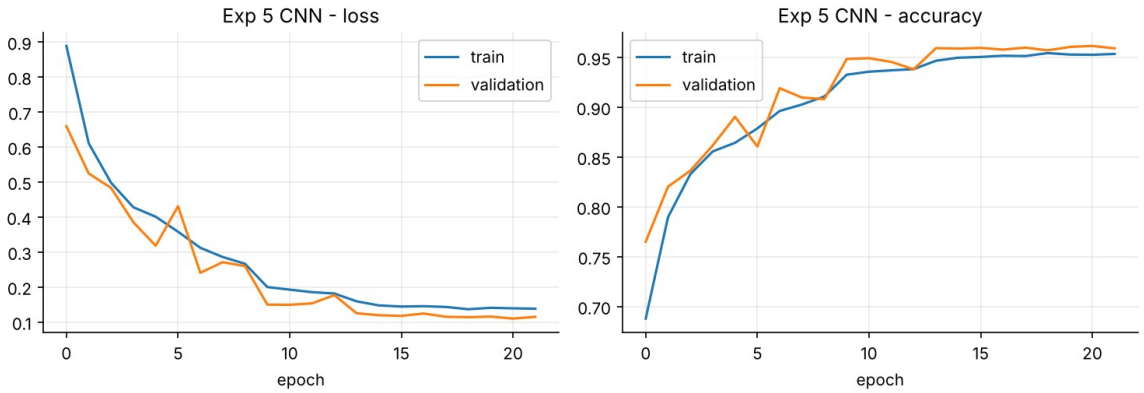


Figure 9: Exp 5 - CNN training and validation curves

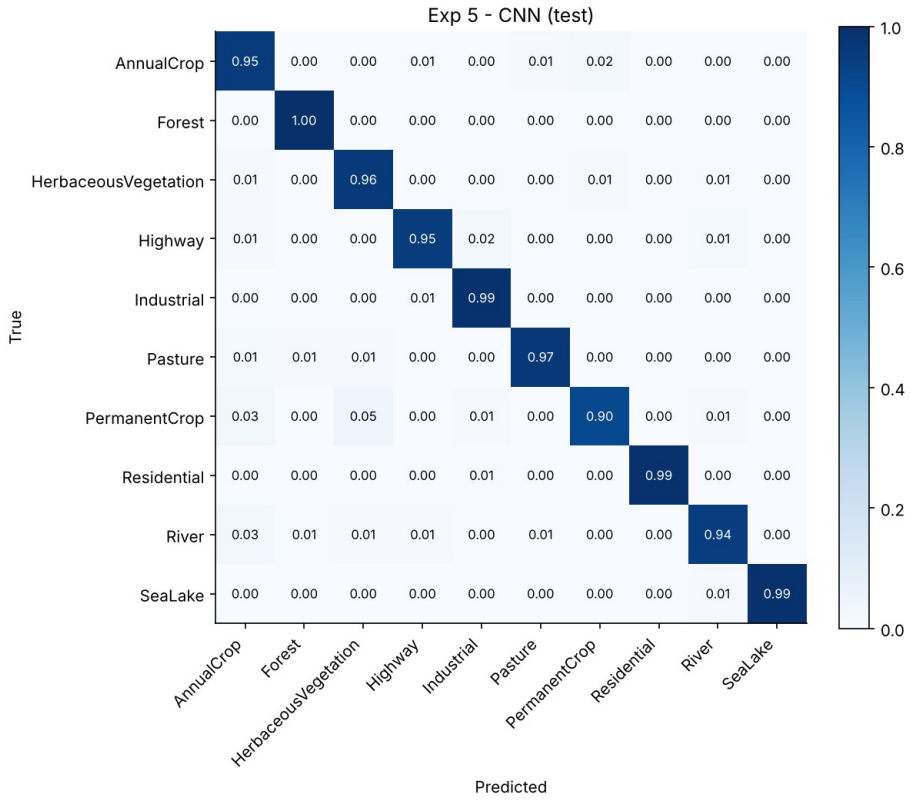


Figure 10: Exp 5 - normalised confusion matrix (test set)

Observation: the hardest classes are PermanentCrop (F1 0.931), AnnualCrop (F1 0.940), River (F1 0.947). PermanentCrop is confused mostly with HerbaceousVegetation and AnnualCrop, which look alike at 10 m per pixel, and some narrow rivers are taken for AnnualCrop field strips. Highway and River are occasionally swapped because both are thin linear features. Forest, Residential and SeaLake reach an F1 of about 0.99.

5.5 Experiment 4 - Visualisation of feature maps

Using the Exp 5 CNN we visualised (a) the 32 learned first-layer 3×3 RGB filters, (b) the activation maps of four convolutional layers for sample images, and (c) Grad-CAM heat-maps, computed from the gradient of the predicted class score with respect to the last convolutional layer.

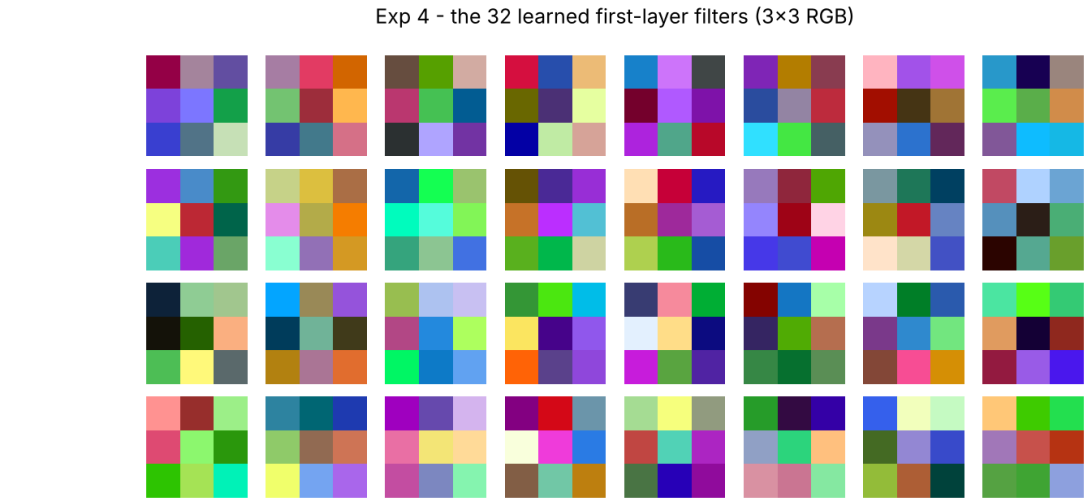


Figure 11: Exp 4 - the 32 first-layer 3×3 filters (RGB weights, min-max scaled per filter)

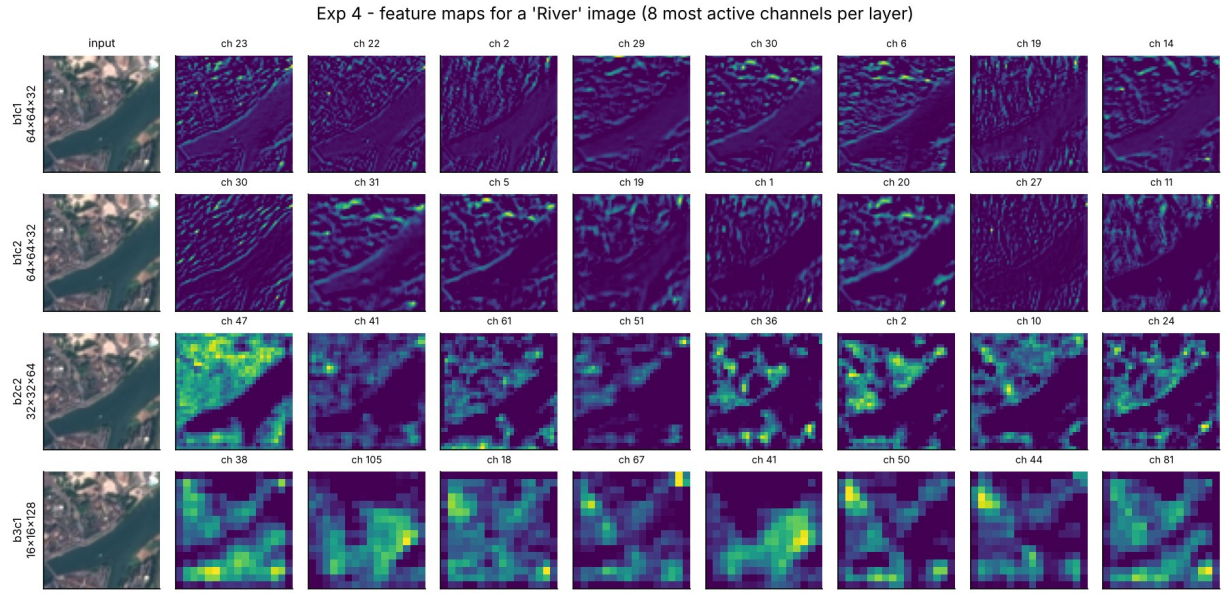


Figure 12: Exp 4 - feature maps for a River image at increasing depth (8 most active channels per layer)

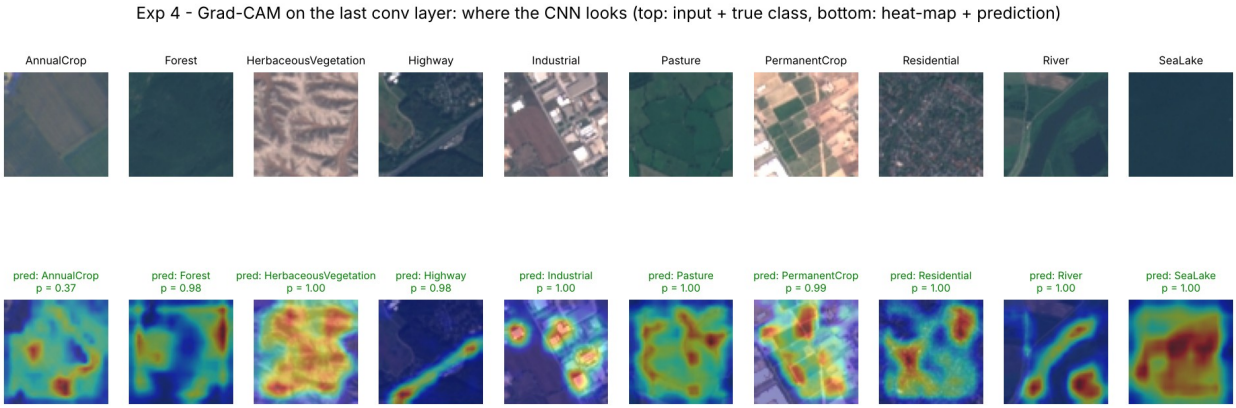


Figure 13: Exp 4 - Grad-CAM: the image regions that drive each prediction

Table 11: Exp 4 - activation statistics on 256 test images

Layer	Output shape	Mean activation	Fraction of zeros
b1c1_relu	64×64×32	0.3886	42.4%

Layer	Output shape	Mean activation	Fraction of zeros
b1c2_relu	64×64×32	0.3025	53.4%
b2c2_relu	32×32×64	0.2601	60.1%
b3c1_relu	16×16×128	0.4381	57.4%

Observation: a 3×3 filter is too small to show a clear pattern on its own, but its effect is visible in the feature maps. In block 1, channels respond to edges and colour boundaries and keep the full 64×64 detail, so the river bank and field borders are traced sharply. By block 2 the maps are coarser (32×32) and more semantic: in the River example one channel fires on all the land and stays silent over the water, which effectively segments the river. The block 3 maps (16×16) respond to larger regions and textures. After ReLU the share of zero activations rises from 42% in the first layer to 60% in block 2, so deeper representations are sparser. Grad-CAM shows the network looking at the right evidence: the road for Highway, the bright roofs for Industrial, the channel for River and the whole water body for SeaLake. This supports trusting the classifier's decisions.

5.6 Experiment 6 - Transfer learning with MobileNetV2

ImageNet-pretrained MobileNetV2 backbones were adapted to EuroSAT. Images are resized from 64×64 to 128×128 inside the model (an earlier run at 96×96 gave 3.5 points lower frozen-feature accuracy) and scaled to [-1, 1]. The new head is GlobalAveragePooling → Dropout(0.3) → Dense(10, softmax). **Phase A (feature extraction):** the backbone is frozen and only the head is trained (backbone features are computed once). **Phase B (fine-tuning):** inverted-residual blocks 10-16 and the final 1×1 convolution are unfrozen (from backbone layer 90 of 154), BatchNorm statistics are kept frozen, and the model is trained end-to-end for up to 12 epochs with Adam at 1e-4 (halved on plateau) and the same augmentation as Exp 5.

Table 12: Exp 6 - transfer learning compared with the scratch CNN

Model	Total params	Frozen acc.	Fine-tuned acc.	Fine-tuned F1	Weights (KB)	Latency (ms)
MobileNetV2 α = 1.0	2,270,794	93.19%	97.58%	0.9749	8870	7.75
MobileNetV2 α = 0.35	423,018	92.12%	95.53%	0.9537	1652	5.64
CNN from scratch (Exp 5)	141,674	-	96.54%	0.9646	553	2.56

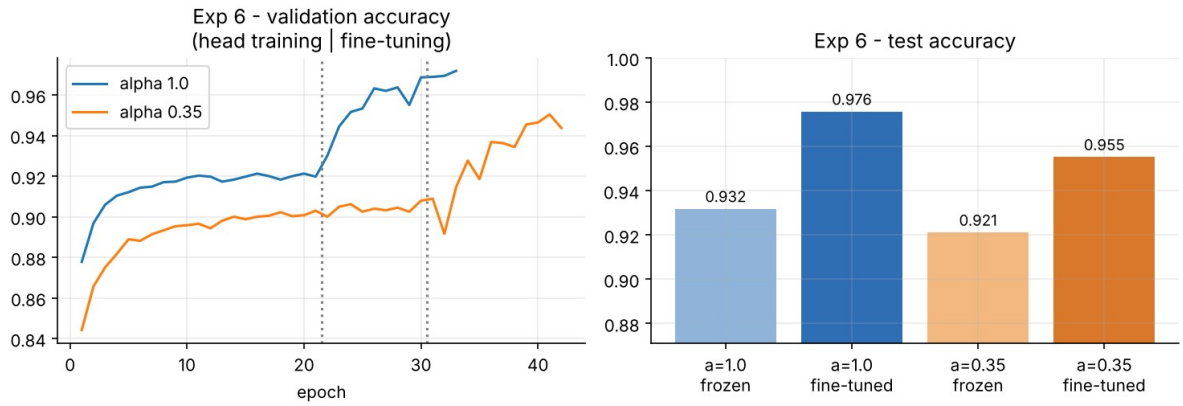


Figure 14: Exp 6 - validation accuracy across both phases, and test accuracy

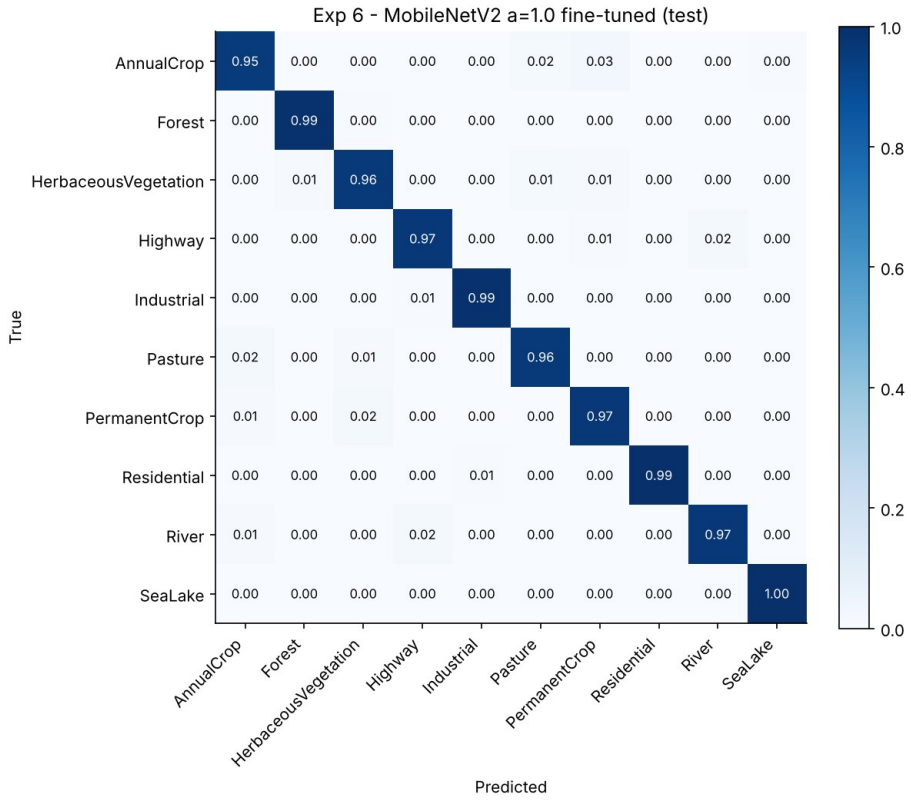


Figure 15: Exp 6 - confusion matrix of the best fine-tuned MobileNetV2

Observation: with the backbone frozen, ImageNet features alone give 93.19% (α = 1.0) and 92.12% (α = 0.35) after only a few minutes of head training. Fine-tuning the top blocks adds 4.4 percentage points for α = 1.0, reaching 97.58%, which is 1.0 points above our scratch CNN. The α = 0.35 model has 5.4× fewer parameters than α = 1.0 and reaches 95.53%. This is the accuracy-versus-size trade-off a low-power PocketQube has to make. Our scratch CNN remains the smallest model (141,674 parameters), which is why it is the one deployed in the browser.

5.7 Experiment 7 - Recurrent neural network

Task: given the last 60 minutes of all 7 telemetry channels, predict battery voltage and battery temperature for each of the next 15 minutes (a 15 × 2 output). Windows are taken every 3 minutes and never cross run boundaries (29,952 training, 7,488 validation and 7,488 test windows). **Model:** SimpleRNN(64, tanh) → Dense(30) → reshape to 15 × 2, with 6,558 parameters. Trained with MSE loss, Adam (2e-3), early stopping and LR reduction (30 epochs).

Baseline: a persistence forecast (the future equals the last value), which is hard to beat over short horizons. The SimpleRNN reaches a voltage MAE of **21.6 mV** against 31.0 mV for persistence, and a temperature MAE of **0.88 °C** against 4.53 °C.

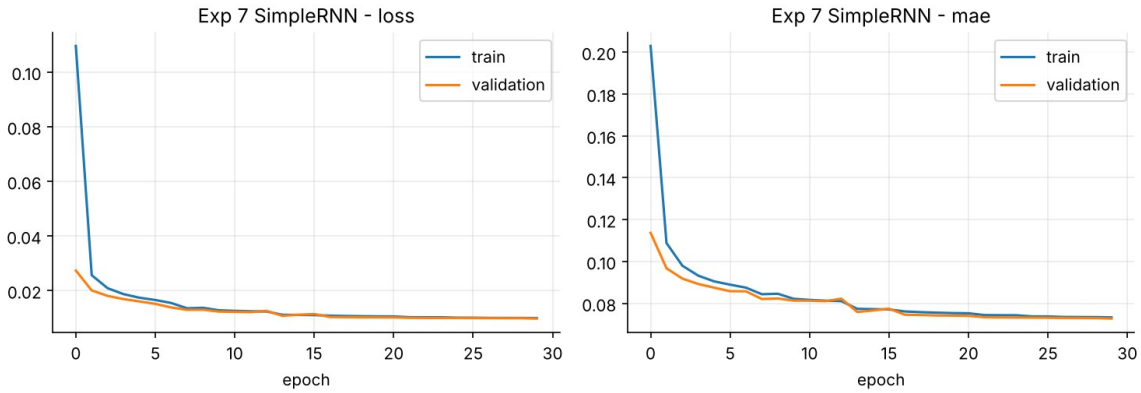


Figure 16: Exp 7 - SimpleRNN training curves (MSE on standardised targets)

5.8 Experiment 8 - Long Short-Term Memory network

The SimpleRNN layer was replaced by an LSTM(64) with otherwise identical settings (20,382 parameters, 30 epochs). The LSTM's gates let it carry information about the orbit phase (time since entering eclipse) and the recent load history across the 60-step window.

Table 13: Exp 7 vs Exp 8 - forecasting error on held-out runs

Model	Params	V MAE	V MAE @ t+15	T MAE (°C)	T MAE @ t+15 (°C)
Persistence	-	31.0 mV	41.3 mV	4.529	8.014
SimpleRNN (Exp 7)	6,558	21.6 mV	24.1 mV	0.882	0.955
LSTM (Exp 8)	20,382	19.0 mV	21.1 mV	0.785	0.814

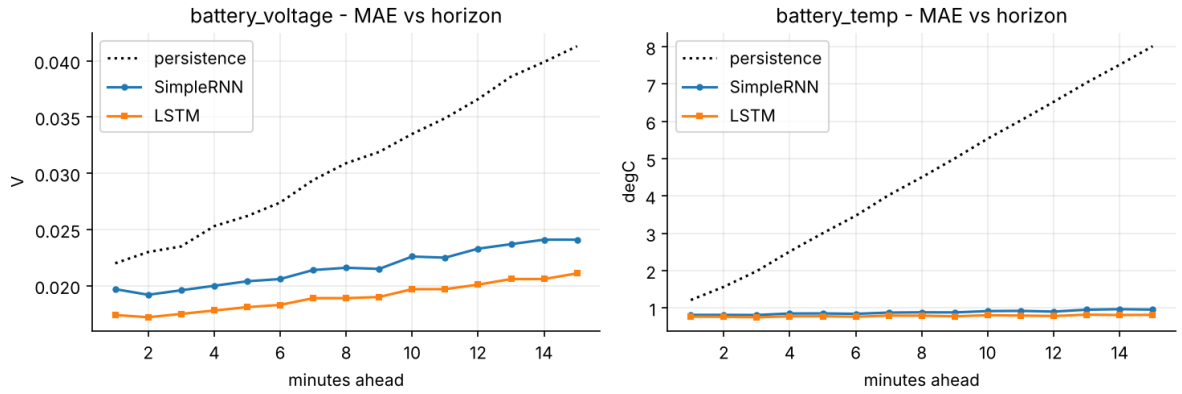


Figure 17: MAE at each horizon from 1 to 15 minutes: persistence, SimpleRNN and LSTM

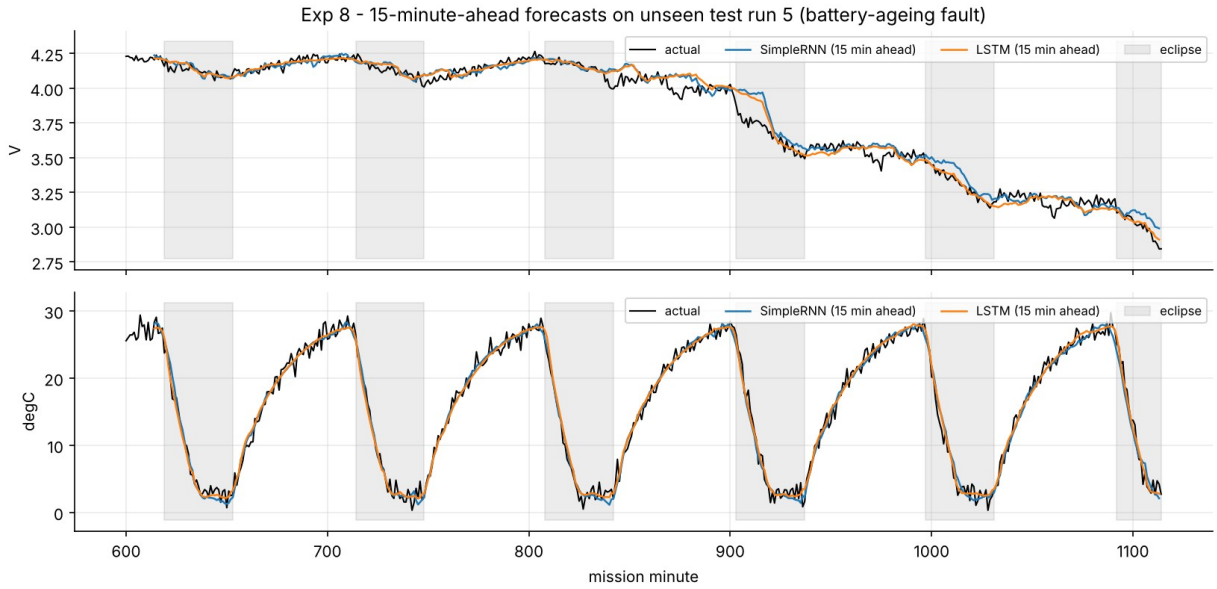


Figure 18: 15-minute-ahead forecasts on an unseen run with a battery-ageing fault (grey bands are eclipses)

Observation: both recurrent models clearly beat persistence. The persistence error grows quickly with the horizon (temperature: 1.21 °C at t+1 but 8.01 °C at t+15), because the satellite keeps crossing between sunlight and eclipse. The learned models know the orbit phase from the input window and anticipate these crossings. Compared with the SimpleRNN, the LSTM's voltage MAE is 12.0% lower and its temperature MAE is 11.0% lower, and the gap widens towards t+15 (voltage 24.1 mV vs 21.1 mV). This is consistent with the LSTM's gates keeping longer-range context that the SimpleRNN's hidden state loses. The sensors themselves have noise with a standard deviation of 18 mV and 0.9 °C, which puts the lowest achievable MAE at about 14 mV and 0.72 °C, so the LSTM is close to this floor. In the example run the forecasts follow the eclipse cycles and the steady decline caused by the ageing cell. The remaining error is concentrated at the sharp voltage drops at eclipse entry late in the fault, which the models anticipate only partially and follow a few minutes late.

6. Comparative Analysis

Table 14: All models side by side (held-out test data)

Task	Model	Params	Weights (KB)	Main metric
Health classification	NumPy MLP (Exp 1)	179	0.7	acc 97.59%, F1 0.9497
Health classification	FFNN (Exp 2)	179	0.7	acc 98.09%, F1 0.957
Health classification	DNN (Exp 3)	12,131	47	acc 98.45%, F1 0.9624
Image classification	CNN scratch (Exp 5)	141,674	553	acc 96.54%
Image classification	MobileNetV2 α1.0 (Exp 6)	2,270,794	8870	acc 97.58%
Image classification	MobileNetV2 α0.35 (Exp 6)	423,018	1652	acc 95.53%
Forecasting (15 min)	SimpleRNN (Exp 7)	6,558	26	V 21.6 mV, T 0.88 °C
Forecasting (15 min)	LSTM (Exp 8)	20,382	80	V 19.0 mV, T 0.78 °C

- For telemetry health, all three networks exceed 97.6% accuracy on unseen orbits. The remaining errors are borderline readings next to a limit, where the sensor noise makes the true class ambiguous.
- Depth helps the health task (Exp 3 depth study) and a lot for images: a 141,674-parameter CNN reaches 96.54%, and ImageNet transfer learning reaches 97.58%.
- The models used on the dashboard are all small (float32 weights): DNN 47 KB, CNN 553 KB, LSTM 80 KB. A desktop CPU runs the CNN in 2.56 ms per image, and the browser version takes about 0.15 s. MobileNetV2 α1.0 needs 8870 KB, which is 16× more than our CNN.
- The LSTM is the better forecaster (Exp 8 vs Exp 7) for a modest increase in parameters (6,558 → 20,382).

7. Web Dashboard and Deployment

The demonstration is a static web application (web/) deployed on Vercel from the GitHub repository. All inference runs in the visitor's browser using the exported weights, so there is no server and no Python is needed at run time:

- **Live health monitor (Exp 3 DNN):** sliders for all 7 telemetry inputs, plus scenario presets (nominal, eclipse, SSTV downlink, latch-up, solar loss, heater failure, battery ageing). Shows the predicted state and class probabilities.
- **Orbit replay with forecast (Exp 8 LSTM):** plays back a held-out run that contains a battery-ageing fault. Every minute the LSTM forecasts the next 15 minutes of voltage and temperature, and the DNN labels the current state, which is shown next to the ground truth.
- **Earth image classifier (Exp 5 CNN, Exp 4 feature maps):** click any of 30 held-out test images, or upload an image. The CNN's top-3 classes and its first-layer feature maps are computed in the browser.
- **Experiments and results:** cards for all eight lab experiments, the comparison tables and every plot.

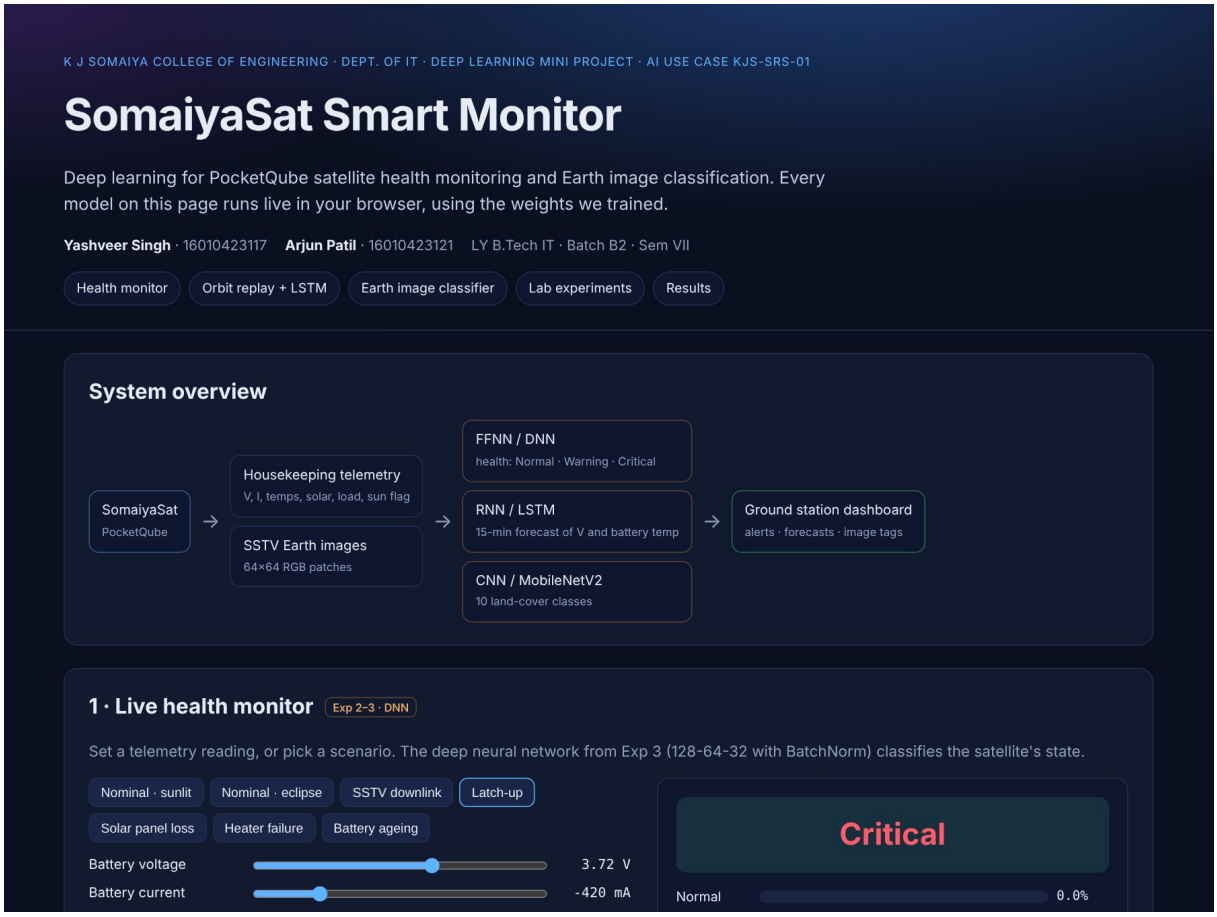


Figure 19: The deployed dashboard

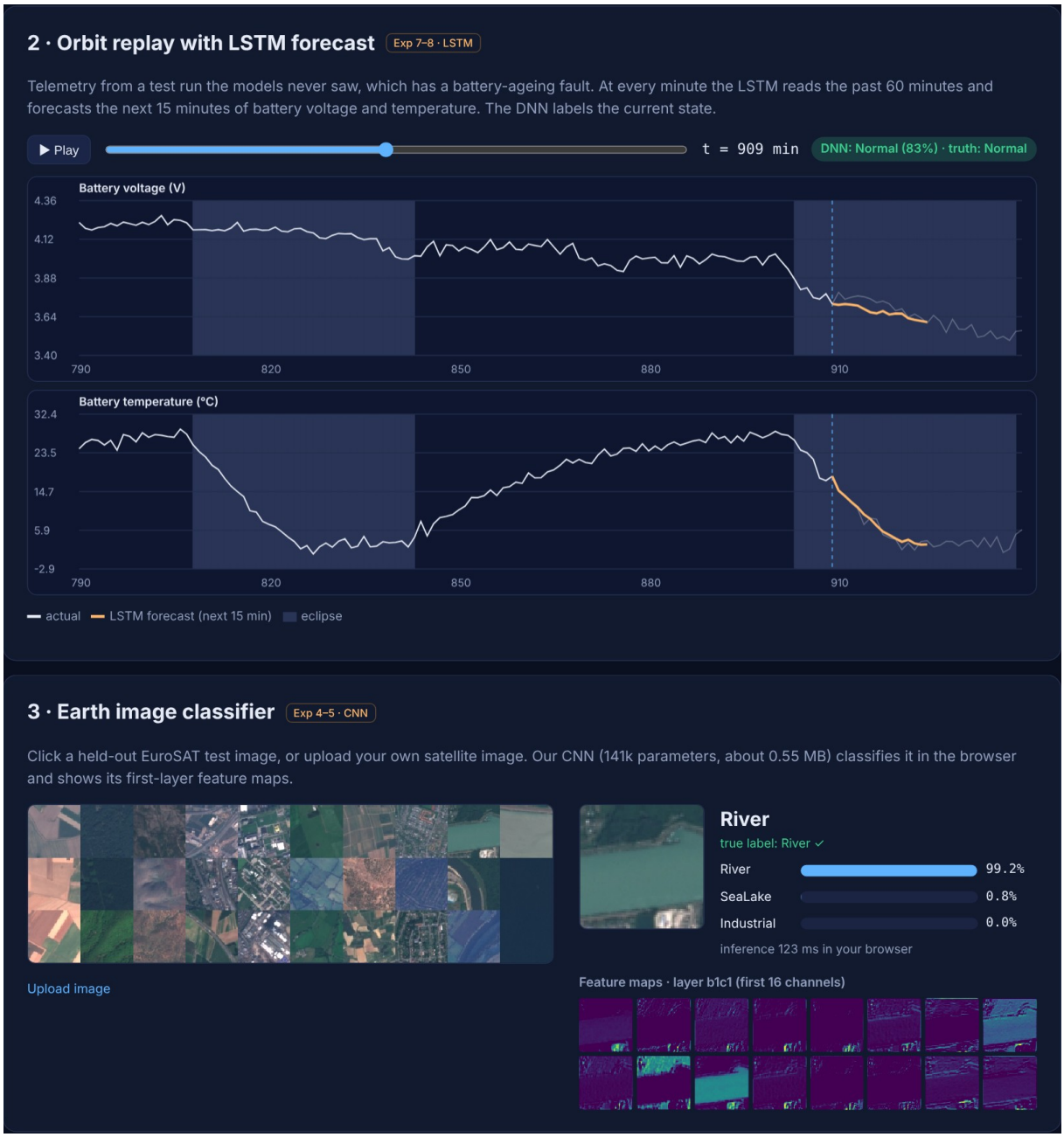


Figure 20: Dashboard - orbit replay with LSTM forecast and the Earth image classifier

Correctness of the browser models: the JavaScript CNN (with BatchNorm folded into the convolution weights), the DNN and the LSTM were checked against Keras on the same inputs and agree to within floating-point rounding (see Appendix A).

Deployment steps: (1) push the repository to GitHub; (2) in Vercel choose 'Add New → Project', import the repository and deploy. vercel.json already sets the output directory to web/ with no build step.

8. Conclusion and Future Scope

We built SomaiyaSat Smart Monitor, an end-to-end deep learning system for the KJS-SRS-01 PocketQube mission that uses all eight Deep Learning lab experiments. Health classifiers built from back-propagation (Exp 1), feed-forward (Exp 2) and deep (Exp 3) networks label telemetry with up to 98.45% accuracy and never mistake a Critical state for a Normal one (Exp 3 DNN). RNN and LSTM forecasters (Exp 7, 8) predict battery voltage and temperature 15 minutes ahead, and the LSTM achieves 19.0 mV and 0.78 °C MAE. For Earth images, a 141,674-parameter CNN (Exp 5) reaches 96.54% on EuroSAT. Its feature maps and Grad-CAM (Exp 4) show that it focuses on meaningful structures, and MobileNetV2 transfer learning (Exp 6) raises accuracy to 97.58%. All of this runs in a browser dashboard deployed on Vercel. The results show that lightweight deep learning models can support autonomous monitoring and smarter data handling in small, student-built satellites.

Limitations

- The telemetry is simulated. It follows real PocketQube physics, but the models must be re-trained or fine-tuned on actual SomaiyaSat downlink data once it is available.
- EuroSAT images (10 m/pixel, Sentinel-2) differ from SomaiyaSat camera and SSTV frames in resolution, colour response and compression, so domain adaptation will be needed.
- The health labels come from fixed operator limits. Novel faults outside these rules would need an unsupervised method (e.g. alerts on forecast error).

Future scope

- Raise alerts when the LSTM forecast deviates from the observed telemetry, to detect faults the rules do not cover.
- Quantise the CNN or MobileNetV2-α0.35 to int8 (TensorFlow Lite) for an on-board microcontroller, so images can be filtered before downlink.
- Simulate SSTV transmission noise during training, to make the image classifier robust to it.
- Connect the dashboard to the live ground-station decoder.

9. References

[1] J. Patterson and A. Gibson, Deep Learning: A Practitioner's Approach, O'Reilly Media, 2017.

[2] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning, MIT Press, 2016.

[3] P. Helber, B. Bischke, A. Dengel and D. Borth, "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification," IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 12, no. 7, pp. 2217-2226, 2019.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," Proc. IEEE CVPR, 2018.

[5] K. Hundman, V. Constantinou, C. Laporte, I. Colwell and T. Soderstrom, "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding," Proc. ACM KDD, 2018.

[6] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," Proc. IEEE ICCV, 2017.

[7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[8] N. Buduma, Fundamentals of Deep Learning, O'Reilly Media, 2017.

Appendix A - Project Structure and How to Run

Table 15: Repository layout

Path	Contents
src/telemetry_sim.py	PocketQube telemetry simulator (data/telemetry.csv)
src/exp1_backprop.py ... exp8_lstm.py	One script per lab experiment
src/export_web.py	Exports weights, gallery and results to web/
results/metrics, results/plots	All metrics (JSON) and figures used in this report
models/	Trained Keras models (.keras)
web/	Dashboard: index.html, js/nn.js (inference), js/app.js, data/, img/
report/	This report (DOCX/PDF) and its generator
run_all.sh, requirements.txt, vercel.json	Re-run everything; dependencies; Vercel config

Run: pip install -r requirements.txt; put EuroSAT_RGB.zip in data/; run ./run_all.sh. **View locally:** python -m http.server -d web 8000 and open http://localhost:8000.

Browser-model check: maximum absolute difference between JavaScript and Keras outputs: CNN 1.7e-7 (40 images, top-1 agreement 100%), DNN 2.0e-7, LSTM 8.0e-6.